



Web Technologies

WET2VO

Teil 4: Objektorientiertes PHP

Medientechnik und -design | SS 2026

Modularisierung

Dateiinhalte aufteilen



include & require

vars.inc.php

```
<?php  
$test = "<p>Works!</p>";  
const MESSAGE = <p>Hello World</p>;
```

include.php

```
<?php  
include "vars.inc.php";  
echo $test; // <p>Works!</p>  
echo MESSAGE; // <p>Hello World</p>
```

include

Mehrfache Einbindung
E_WARNING im Fehlerfall

require

Mehrfache Einbindung
Error Exception im Fehlerfall

include_once

Einmalige Einbindung
E_WARNING im Fehlerfall

require_once

Einmalige Einbindung
Error Exception im Fehlerfall

Objektorientierung

PHP: The Modern Way

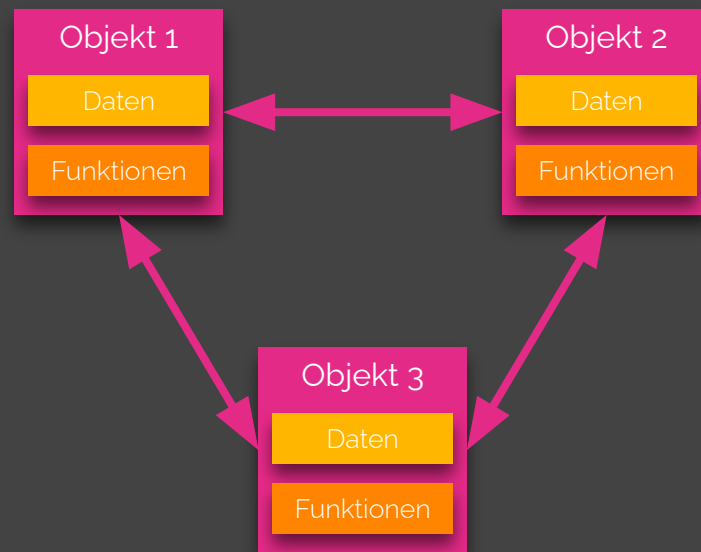
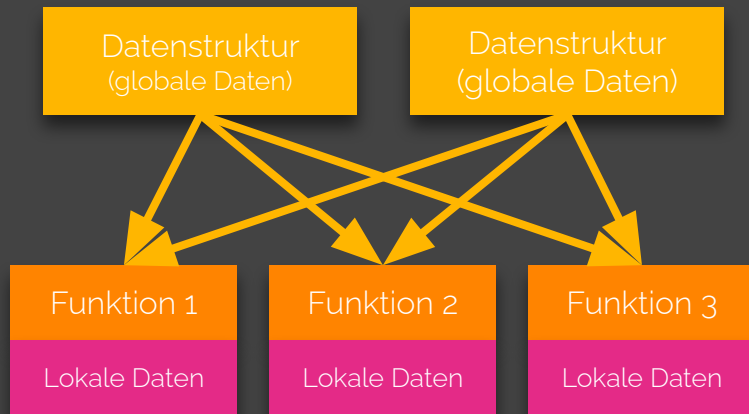


Warum

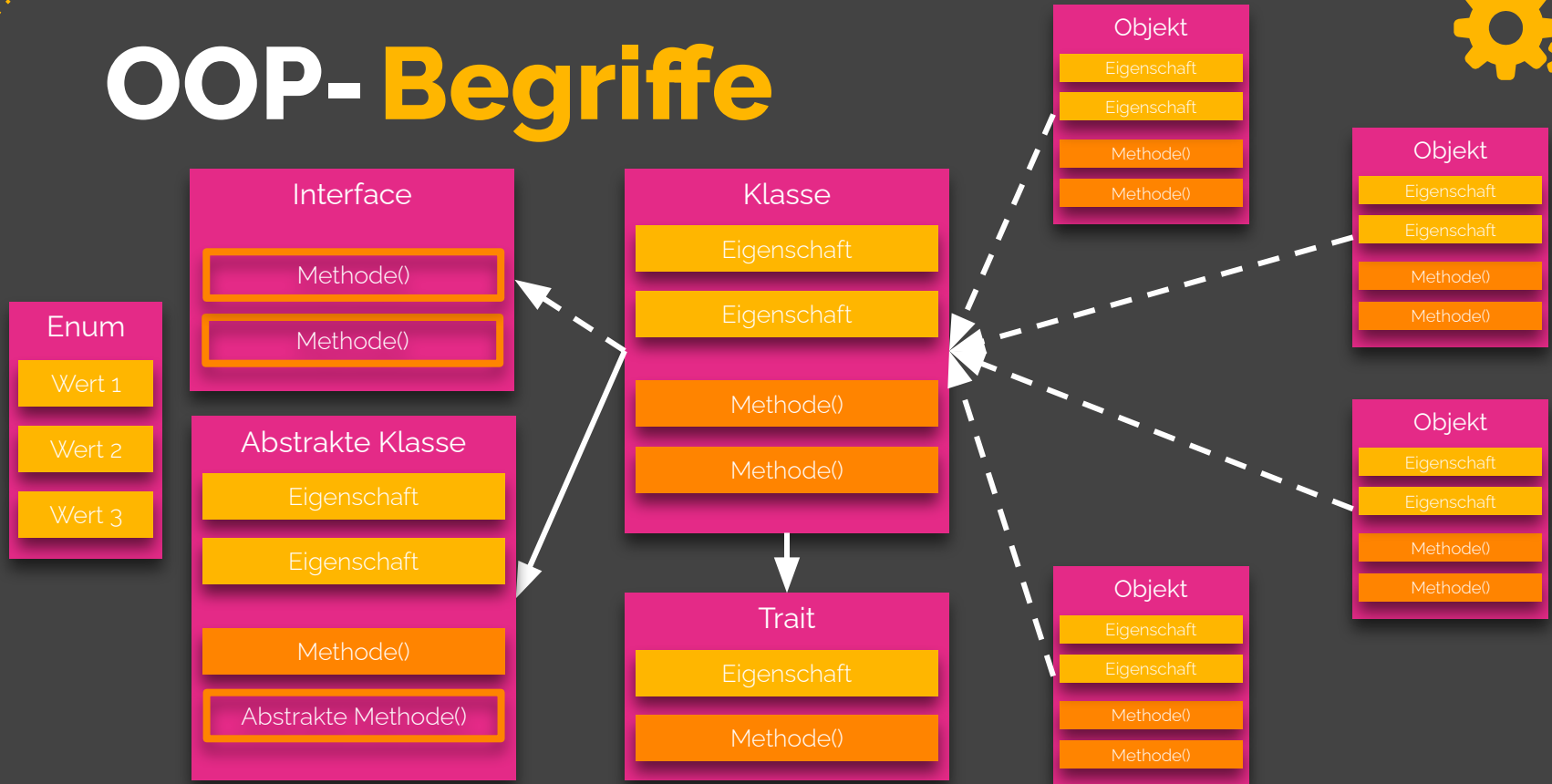
Objektorientierung

Prozedural

Objektorientiert



OOP-Begriffe





Objekte erzeugen & Zugriff

```
<Objektvariable> = new <Name der Klasse()>  
<Objektvariable>-><Name der Eigenschaft>  
<Objektvariable>-><Name der Methode()>
```

```
$rect1 = new Rectangle();  
$rect2 = new Rectangle(45, 60, 110, 120, $green);
```

```
$rect1->x1 = 5;  
$rect1->y1 = 5;  
$rect1->x2 = 10;  
$rect1->y2 = 10;  
$rect1->color = $red;  
$rect1->draw();
```

Klassen & Eigenschaften



```
class Rectangle
{
    public $x1; // ggf. private, protected
    public $y1;
    public $x2;
    public $y2;
    public $color;

    // ...
}
```


Typed Properties



```
class Rectangle
{
    public int $x1; // ggf. private, protected
    public int $y1;
    public int $x2;
    public int $y2;
    public string $color;

    // ...
}
```

Asymmetric Visibility



```
class Rectangle
{
    public int $x1; // ggf. private, protected
    public int $y1;
    public int $x2;
    public int $y2;
    public private(set) string $color;

    // ...
}
```

Klassen & Methoden



```
class Rectangle
{
    public int $x1; public int $y1; public int $x2; public int $y2;
    public string $color;
```

```
    public function move(int $dx, int $dy): void
```

```
    {
```

```
        $this->x1 += $dx; $this->y1 += $dy;
```

```
        $this->x2 += $dx; $this->y2 += $dy;
```

```
    }
```

```
    public function getX1(): int
```

```
    {
```

```
        return $this->x1;
```

```
    }
```

```
    public function draw(): void
```

```
    { // ... }
```

```
}
```

„Greife auf die Eigenschaft
(Exemplarvariable, Member-Variable) `$x1`
im aktuellen **Objekt** zu.“

Property Hooks



```
class Rectangle
{
    public int $x1 {
        get {
            return $this->x1;
        }
    }
    public int $y1;
    public int $x2;
    public int $y2;
    public private(set) string $color;
    // ...
}
```

```
// Zugriff
$rect1->x1;
```

```
class Rectangle
{
    public int $x1;
    public int $y1;
    public int $x2;
    public int $y2;
    public string $color;

    public function getX1(): int
    {
        return $this->x1;
    }
}
```

```
// Zugriff
$rect1->getX1();
```

Konstruktor & Destruktor



```
class Rectangle
{
    public int $x1; public int $y1; public int $x2; public int $y2;
    public string $color;

    public function __construct(int $x1 = 0, int $y1 = 0, int $x2 = 0, int $y2 = 0,
                                string $color = "white")
    {
        $this->x1 = $x1;
        $this->y1 = $y1;
        $this->x2 = $x2;
        $this->y2 = $y2;
        $this->color = $color;
    }

    public function __destruct()
    {
        echo "Destructor called!";
    }
}
```



Constr. Property

Ab PHP 8: Zugriffsmodifikator vor Parameter

Promotion

keine Eigenschaften und separate Initialisierung mehr nötig

```
class Rectangle
{
    public function __construct(
        public int $x1 = 0,
        public int $y1 = 0,
        public int $x2 = 0,
        public int $y2 = 0,
        public string $color = "white"
    ) {}

    // ...
}
```

```
class Rectangle
{
    public function __construct(
        public int $x1 = 0 {
            get {
                return $this->x1;
            }
        },
        public int $y1 = 0,
        public int $x2 = 0,
        public int $y2 = 0,
        public string $color = "white"
    ) {}
}
```

Statische Variablen



```
class Rectangle
{
    // ...
    public static string $version = "1.0.0";
    // ...

    public function getVersion(): string {
        return self::$version;
    }
    // ...
}
```

Scope Resolution Operator

Paamayim Nekudotayim
(zweimal Doppelpunkt)

„Greife auf die statische Eigenschaft `$version`
in der aktuellen **Klasse** (`Rectangle`) zu“

```
$rect1 = new Rectangle();
echo Rectangle::$version; // 1.0.0
echo $rect1->getVersion(); // 1.0.0
echo $rect1->version; // NOTICE & WARNING!
```

Statische Methoden



```
class Rectangle
{
    // ...
    public static string $version = "1.0.0";
    // ...

    public static function printVersion(): void {
        echo self::$version;
    }
    // ...
}

$rect1 = new Rectangle();
Rectangle::printVersion(); // 1.0.0
$rect1->getVersion(); // 1.0.0 - eher nicht verwenden
```




Konstanten

```
class Rectangle
{
    // ...
    public const string TYPE = "Rectangle";
    // ...

    public function getType(): string
    {
        return self::TYPE;
    }
    // ...
}
```

„Greife auf die Konstante TYPE in der aktuellen **Klasse** (Rectangle) zu“

```
$rect1 = new Rectangle();
echo Rectangle::TYPE; // Rectangle
echo $rect1->getType(); // Rectangle
echo $rect1->TYPE; // FEHLER!
```



Nur lesende

Eigenschaften

```
class Rectangle
{
    public function __construct(
        public readonly int $id,
        public int $x1 = 0,
        public int $y1 = 0,
        public int $x2 = 0,
        public int $y2 = 0,
        public string $color = "white"
    ) {
    }
    // ...
}
```

```
$rect1 = new Rectangle(1);
$rect1->id = 3; // FEHLER!
```

Readonly Klassen



```
class Post
{
    public function __construct(
        public readonly string $title,
        public readonly string $author,
        public readonly string $content
    ) {
    }
}
```



```
readonly class Post
{
    public function __construct(
        public string $title,
        public string $author,
        public string $content
    ) {
    }
}
```

Interfaces



GeometricComponentInterface.php

```
interface GeometricComponentInterface
{
    public function move(int $dx,
                        int $dy): void;
    public function draw(): void;
}
```

Rectangle.php

```
require "GeometricComponentInterface.php";

class Rectangle implements GeometricComponentInterface
{
    // ...

    public function move(int $dx, int $dy): void
    {
        $this->x1 += $dx; $this->y1 += $dy;
        $this->x2 += $dx; $this->y2 += $dy;
    }
    // ...

    public function draw(): void
    {
        // ...
    }
}
```

Attribute



```
Rectangle.php
require "GeometricComponentInterface.php";

class Rectangle implements GeometricComponentInterface
{
    #[Override]
    public function move(int $dx, int $dy): void
    {
        $this->x1 += $dx; $this->y1 += $dy;
        $this->x2 += $dx; $this->y2 += $dy;
    }

    #[Override]
    public function draw(): void
    {
        // ...
    }
}
```

Vererbung



```
require "Rectangle.php";
```

```
class Box extends Rectangle  
{
```

```
    public int $depth;
```

```
    #[Override]
```

```
    public function draw(): void  
{
```

```
        // Vorderseite zeichnen (Rechteck)
```

```
        parent::draw();
```

```
        // andere Seiten zeichnen
```

```
        // ...
```

```
    }
```

```
}
```

Neue Eigenschaft!
\$x1, \$x2 etc. gelten auch für Box

draw() wird mit neuer Funktionalität
überschrieben
getX1(), bzw. der get-Property Hook und
move() bleiben auch in Box gleich



Typüberprüfung

```
class Human { ... }  
class Employee extends Human { ... }  
class Animal { ... }  
  
$obj = new Employee();  
if ($obj instanceof Employee) { // true  
    echo "Object is Employee";  
}  
if ($obj instanceof Human) { // true  
    echo "Object is Human";  
}  
if ($obj instanceof Animal) { // false  
    echo "Object is Animal";  
}
```

Final: Vererbung stoppen



Finale Klasse

```
final class NoInheritance
{
    public function foo(): void
    {
        // ...
    }
}
```

Finale Methode und Konstante

```
class CantOverrideEverything
{
    public const int CAN_OVERRIDE = 1;
    final public const int CANNOT_OVERRIDE = 2;

    public function canOverride(): void
    {
        // ...
    }

    final public function cantOverride(): void
    {
        // ...
    }
}
```


Abstrakte Klassen



AbstractClass.php

```
abstract class AbstractClass
{
    abstract protected function getValue(): string;

    public function printOut(): void
    {
        echo $this->getValue();
    }
}
```

ConcreteClass.php

```
require "AbstractClass.php";

class ConcreteClass extends AbstractClass
{
    #[Override]
    protected function getValue(): string
    {
        return "ConcreteClass";
    }
}
```

Traits



```
Person.php
class Person
{
    // Implementierung
}
```

```
Car.php
class Car
{
    // Implementierung
}
```

Beide Klassen sollen die Möglichkeit haben, ihre **aktuelle Position** zu speichern und auszugeben.

1. Code in beiden Klassen **unabhängig implementieren**.
2. **Interface** (z.B. **LocatableInterface**) erstellen und für beide Klassen verwenden.
3. **Abstrakte Elternklasse** (z.B. **AbstractLocatable**) erstellen und beide davon ableiten.
4. **Trait** definieren (z.B. **LocatableTrait**), der die Funktionalität definiert und in beiden Klassen eingebunden wird.

Traits implementieren



LocatableTrait.php

```
trait LocatableTrait
{
    public ?float $latitude = null;
    public ?float $longitude = null;

    public function setPosition(array $pos): void
    {
        $this->latitude = $pos[0];
        $this->longitude = $pos[1];
    }

    public function getPosition(): array
    {
        return [$this->latitude, $this->longitude];
    }
}
```

Traits verwenden



Person.php

```
require "LocatableTrait.php";
```

```
class Person
```

```
{
```

```
    use LocatableTrait;
```

```
    // Implementierung
```

```
}
```

```
$pers = new Person();
```

```
$pers->setPosition([48.368201, 14.514065]);
```

```
echo "<p>Latitude: {$pers->latitude}°</p>";
```

```
echo "<p>Longitude: {$pers->longitude}°</p>";
```

Car.php

```
require "LocatableTrait.php";
```

```
class Car
```

```
{
```

```
    use LocatableTrait;
```

```
    // Implementierung
```

```
}
```

Enums



ExportFormat.php

```
enum ExportFormat
{
    case XML;
    case JSON;
    case PDF;
}
```

Aufruf

```
$exporter = new ProductExporter();
$exporter->export(ExportFormat::JSON);
```

ProductExporter.php

```
class ProductExporter
{
    public function export(ExportFormat $format): void
    {
        match ($format) {
            ExportFormat::XML => $this->exportToXML(),
            ExportFormat::JSON => $this->exportToJSON(),
            ExportFormat::PDF => $this->exportToPDF(),
        };
    }

    private function exportToXML(): void
    {
        // Export to XML
    }

    // ...
}
```

Werte & Backed Enums



ExportFormat.php

```
echo ExportFormat::XML->name; // XML
echo ExportFormat::JSON->name; // JSON
echo ExportFormat::PDF->name; // PDF
```

BackedExportFormat.php

```
enum BackedExportFormat: int
{
    case XML = 1;
    case JSON = 2;
    case PDF = 3;
}
```

```
echo BackedExportFormat::XML->name; // XML
echo BackedExportFormat::XML->value; // 1
```

Exceptions



```
try {  
    throw new Exception("General error!");  
    echo "This output isn't shown anymore.";  
}  
catch (Exception $e) {  
    echo "Exception: " . $e->getMessage();  
}  
  
echo "Continue after exception handling.";
```

Eigene Exceptions



```
class MyException extends Exception
{
    // ...
}
```

Exceptions der Standard-PHP-Library (SPL):

- LogicException (extends Exception)
 - BadFunctionCallException
 - BadMethodCallException
 - DomainException
 - InvalidArgumentException
 - LengthException
 - OutOfRangeException
- RuntimeException (extends Exception)
 - OutOfBoundsException
 - OverflowException
 - RangeException
 - UnderflowException
 - UnexpectedValueException

Namespaces

Namensgleichheiten vermeiden



Motivation

```
class Rectangle  
{  
    // ...  
}
```

```
class Person  
{  
    // ...  
}
```

```
class Car  
{  
    // ...  
}
```

Sind diese Klassen wiederverwendbar mit anderem oder in anderem PHP-Code?

Nur, wenn dieselben Klassennamen nicht vorkommen!

Die Lösung: Namespaces



```
namespace VendorNamespace\SubNamespace1\SubNamespace2;
```

Lecture.php

```
<?php
```

```
namespace Fhooe\Mtd\WebTechnologies;
```

```
class Lecture
```

```
{
```

```
    // ...
```

```
}
```

Exercise.php

```
<?php
```

```
namespace Fhooe\Mtd\WebTechnologies;
```

```
class Exercise
```

```
{
```

```
    // ...
```

```
}
```

Objekte **anlegen**



```
require "Fhooe/Mtd/WebTechnologies/Lecture.php";  
require "Fhooe/Mtd/WebTechnologies/Exercise.php";  
  
use Fhooe\Mtd\WebTechnologies\Exercise;  
  
$lecture = new \Fhooe\Mtd\WebTechnologies\Lecture();  
$exercise = new Exercise();
```

Klassen im globalen NS



Lecture.php

```
<?php
```

```
namespace Foooe\Mtd\WebTechnologies;
```

```
class Lecture
```

```
{
```

```
    // ...
```

```
    $exception = new \Exception();
```

```
    // ...
```

```
}
```

NS und Ordnerstruktur



Namespace: `Fhooe\Mtd\WebTechnologies`

Datei: `Lecture.php`

Verzeichnisstruktur:

`Fhooe`

└─ `Mtd`

└─ `WebTechnologies`

└─ `Lecture.php`

Autoloading

Die Grundlagen

Einfacher Autoloader



```
function autoload(string $fullyQualifiedClassName): void
{
    $fullyQualifiedClassName = ltrim($fullyQualifiedClassName, "\\"); // Backslash am Anfang entf.
    $filePath = "";

    if ($lastBackslashPosition = strrpos($fullyQualifiedClassName, "\\")) { // Letzter Backslash
        $namespace = substr($fullyQualifiedClassName, 0, $lastBackslashPosition); // Namespace
        $shortClassName = substr($fullyQualifiedClassName, $lastBackslashPosition + 1); // Klasse
        $filePath = str_replace("\\", DIRECTORY_SEPARATOR, $namespace) . DIRECTORY_SEPARATOR;
    } else {
        $shortClassName = $fullyQualifiedClassName;
    }
    $filePath .= "$shortClassName.php"; // Pfad = Namespace + Klasse

    if (file_exists($filePath)) {
        require $filePath;
    }
}

spl_autoload_register(autoload(...));
```


Objekte **anlegen**



```
require "autoload.php";
```

```
use Fhooe\Mtd\WebTechnologies\Exercise;
```

```
$lecture = new \Fhooe\Mtd\WebTechnologies\Lecture();
```

```
$exercise = new Exercise();
```



Nächstes Mal in WET2VO

Komponenten und Standards

Credits

- Keine Referenzen in diesem Foliensatz.

